TRI & FUSION DE TABLES

Tri d'une table en python

Lorsqu'on manipule des données en tables, il est fréquent de vouloir les **trier**. Le langage python offre quelques possibilités de tri:

- méthode `sort` des tableaux;
- fonction `sorted` de la librairie standard *-qui sera privilégiée ici-*.

Quels objets peuvent être comparés ?

Python sait comparer évidemment des **nombres** et des **chaines de caractères**, mais aussi des **p-uplets**. Voici quelques exemples:

```
# Comparaison de chaine
print("'abc' > 'aac': ", 'abc' > 'aac')
```

```
'abc' > 'aac':  True
```

```
# Comparaison de p-uplets
print('(5,4) > (5,3): ', (5, 4) > (5, 3))
print('(5,4,6) > (5,4,8): ', (5, 4, 6) > (5, 4, 8))
print("(6, 'c') < (5, 'c'): ", (6, 'c') < (5, 'c'))
print("(6, 'c') < (6, 'e'): ", (6, 'c') < (6, 'e'))
```

```
(5,4) > (5,3):  True
(5,4,6) > (5,4,8):  False
(6, 'c') < (5, 'c'):  False
(6, 'c') < (6, 'e'):  True
```

Qu'en est-il des p-uplets nommés représentés par des dictionnaires?

```
e1 = {'nom':'bob', 'age':20, 'domaine':'ia'}
e2 = {'nom':'alice', 'age':23, 'domaine':'algorithmique'}
print('e1 < e2: ', e1 < e2)
```

```
---------------------------------------------------------------------------

TypeError                                 Traceback (most recent call last)

<ipython-input-3-7ea8c292dfb3> in <module>
      1 e1 = {'nom':'bob', 'age':20, 'domaine':'ia'}
      2 e2 = {'nom':'alice', 'age':23, 'domaine':'algorithmique'}
----> 3 print('e1 < e2: ', e1 < e2)


TypeError: '<' not supported between instances of 'dict' and 'dict'
```

Les p-uplets nommés implémentés en 1ère par des dictionnaires ne peuvent pas être comparés.

Comment utiliser la fonction 'sorted' de la librairie standard?

La fonction `sorted` prend en paramètre obligatoire un *itérable*, comme un tableau ou un tuple par exemple, et renvoie un nouveau tableau trié par **ordre croissant**.

Application directe

L'une des deux instructions suivantes provoque une erreur. Laquelle et pourquoi?

```
t1 = sorted(8, 56, 9, 2, 15)
t2 = sorted((8, 56, 9, 2, 15))
```

Que renvoie l'instruction correcte?

La fonction `sorted` possède un paramètre optionnel `key` qui est utile lorsque python ne sait pas comparer des objets comme les dictionnaires. On affecte à ce paramètre le nom de la fonction chargée d'indiquer la clé suivant laquelle on trie.

Exemple

Soit à ranger le tableau `t` suivant par ordre croissant d'âge:

```python
t = [{'nom':'bob', 'age':24, 'domaine':'IA'},
     {'nom':'alex', 'age':23, 'domaine':'web'},
     {'nom':'sara', 'age':21, 'domaine':'web'},
     {'nom':'alice', 'age':23, 'domaine':'algorithmique'},
     {'nom':'lisa', 'age':23, 'domaine':'robotique'}]

#On définit une fonction qui renvoie un critère de tri
def suivant_age(dico):
    return dico['age']

#On appelle 'sorted' en affectant à 'key' le nom de la fonction de tri
t_trie = sorted(t, key=suivant_age)

#Affichage
for e in t_trie:
    print(e)
```

```
{'nom': 'sara', 'age': 21, 'domaine': 'web'}
{'nom': 'alex', 'age': 23, 'domaine': 'web'}
{'nom': 'alice', 'age': 23, 'domaine': 'algorithmique'}
{'nom': 'lisa', 'age': 23, 'domaine': 'robotique'}
{'nom': 'bob', 'age': 24, 'domaine': 'IA'}
```

On peut vouloir rajouter un deuxième critère de tri, en cas d'égalité par exemple. Il suffit alors de placer ce deuxième critère dans un tuple (que python sait trier!) renvoyé par la fonction.

```python
t = [{'nom':'bob', 'age':24, 'domaine':'IA'},
     {'nom':'alex', 'age':23, 'domaine':'web'},
     {'nom':'sara', 'age':21, 'domaine':'web'},
     {'nom':'alice', 'age':23, 'domaine':'algorithmique'},
     {'nom':'lisa', 'age':23, 'domaine':'robotique'}]

#On définit une fonction qui renvoie le critère de tri
def suivant_age_puis_nom(dico):
    return (dico['age'], dico['nom'])

#On appelle 'sorted' en affectant à 'key' le nom de la fonction de tri
t_trie = sorted(t, key=suivant_age_puis_nom)

#Affichage
for e in t_trie:
    print(e)
```

```
{'nom': 'sara', 'age': 21, 'domaine': 'web'}
{'nom': 'alex', 'age': 23, 'domaine': 'web'}
{'nom': 'alice', 'age': 23, 'domaine': 'algorithmique'}
{'nom': 'lisa', 'age': 23, 'domaine': 'robotique'}
{'nom': 'bob', 'age': 24, 'domaine': 'IA'}
```

La fonction `sorted` possède un dernier paramètre optionnel: `reverse`. On lui donnera la valeur `True` si on veut un classement décroissant.

Application directe

Modifier le code précédent pour trier `t` par domaine, puis par nom en cas d'égalité.

```
t = [{'nom':'bob', 'age':24, 'domaine':'IA'},
     {'nom':'alex', 'age':23, 'domaine':'web'},
     {'nom':'sara', 'age':21, 'domaine':'web'},
     {'nom':'alice', 'age':23, 'domaine':'algorithmique'},
     {'nom':'lisa', 'age':23, 'domaine':'robotique'}]

def suivant_domaine_puis_nom(e):
    #######A compléter#########
    return ...

#Décommentez la ligne ci-dessous et complétez!
#t_trie = sorted(..., ...)

#Affichage
for e in t_trie:
    print(e)
```

Fusion

L'objectif est de réunir dans une seule table les données de deux tables existantes. On examinera deux cas simples:

- tables ayant les mêmes attributs;
- tables ayant au moins un attribut en commun

```
# les tables de travail
import csv
from pprint import pprint


# table des prénoms d'enfants nés à Paris en 2020
with open('data/prenoms2020.csv', newline='', encoding='utf8') as f1:
    t_2020 = [dict(ligne) for ligne in csv.DictReader(f1, delimiter=';')]

# table des prénoms d'enfants nés à Paris en 2021
with open('data/prenoms2021.csv', newline='', encoding='utf8') as f2:
    t_2021 = [dict(ligne) for ligne in csv.DictReader(f2, delimiter=';')]

# table de films
with open('data/Film.csv', newline='', encoding='utf8') as f2:
    t_film = [dict(ligne) for ligne in csv.DictReader(f2, delimiter=',')]

# table d'artistes du cinéma
with open('data/Artiste.csv', newline='', encoding='utf8') as f2:
    t_artiste = [dict(ligne) for ligne in csv.DictReader(f2, delimiter=',')]
```

Comment fusionner des tables ayant les mêmes attributs?

Lorsque des tables possèdent les mêmes attributs, leur fusion ne pose aucun problème. On utilise l'opérateur `+` (*rencontré dans le cours précédent*) sur les deux tableaux existants.

```
# On vérifie que les deux tables ont  les mêmes attributs
print(t_2020[0].keys())
print(t_2021[0].keys())
```

```
dict_keys(['Nombre prénoms déclarés', 'Sexe', 'Annee', 'Prenoms'])
dict_keys(['Nombre prénoms déclarés', 'Sexe', 'Annee', 'Prenoms'])
```

```
t_prenoms = t_2020 + t_2021
print(len(t_2020))
print(len(t_2021))
print(len(t_prenoms))
```

```
1271
1270
2541
```

Comment fusionner des tables ayant des attributs différents?

On se limite au cas simple où les deux tables ont un attribut en commun. L'opération de fusion de table de données est appelée aussi **jointure**.

```
print(t_film[0].keys())
print(t_artiste[0].keys())
```

```
dict_keys(['idFilm', 'titre', 'année', 'genre', 'résumé', 'codePays', 'idArtiste'])
dict_keys(['idArtiste', 'nom', 'prénom'])
```

On remarque que les tables `Film` et `Artiste` ont l'attribut `idArtiste` en commun. Affichons un élément de chaque table.

```
#Affichage de la structure des deux tables
pprint(t_film[0])
print()
pprint(t_artiste[0])
```

```
{'année': '1977',
 'codePays': 'US',
 'genre': 'Aventure',
 'idArtiste': '1',
 'idFilm': '11',
 'résumé': 'Il y a bien longtemps, dans une galaxie très lointaine... La '
           "guerre civile fait rage entre l'Empire galactique et l'Alliance "
           "rebelle. Capturée par les troupes de choc de l'Empereur menées par "
           'le sombre et impitoyable Dark Vador, la princesse Leia Organa '
           'dissimule les plans de l'Étoile Noire, une station spatiale '
           'invulnérable, à son droïde R2-D2 avec pour mission de les remettre '
           'au Jedi Obi-Wan Kenobi. Accompagné de son fidèle compagnon, le '
           "droïde de protocole C-3PO, R2-D2 s'échoue sur la planète Tatooine "
           'et termine sa quête chez le jeune Luke Skywalker. Rêvant de '
           'devenir pilote mais confiné aux travaux de la ferme, ce dernier se '
           'lance à la recherche de ce mystérieux Obi-Wan Kenobi, devenu '
           'ermite au cœur des montagnes désertiques de Tatooine...',
 'titre': 'La Guerre des étoiles'}

{'idArtiste': '1', 'nom': 'Lucas', 'prénom': 'George'}
```

On décide de fusionner les deux tables sur l'attribut `idArtiste` (*qui sera ici le réalisateur du film*). L'algorithme de fusion est le suivant:

```
Pour chaque ligne de la table film:
    Parcourir chaque ligne de la table artiste.
        Si les attributs 'idArtiste' sont les mêmes:
            fusionner les lignes et sauver le résultat.
```

Ecrivons d'abord une fonction qui permet de fusionner deux lignes des deux tables précédentes.

```python
def fusion(f, a):
    """
    Renvoie un dictionnaire, résultat de la fusion des deux lignes e et o;
    f: ligne de la table film (dictionnaire);
    a: ligne de la table artiste (dictionnaire);
    """
    return {'année': f['année'],
            'codePays': f['codePays'],
            'genre': f['genre'],
            'idFilm': f['idFilm'],
            'résumé': f['résumé'],
            'titre': f['titre'],
            'nom': a['nom'],
            'prénom': a['prénom']}
```

On peut implémenter l'algorithme décrit précédemment de deux manières en python:

- en utilisant le type `list` de python et sa méthode `append` (*qui permet d'ajouter un élément à la fin de la liste*);
- en utilisant un tableau construit en compréhension.

```python
# 1ere façon

def jointure_1(tf, ta):
    """ Réalise la fusion entre tf et ta; renvoie un tableau de dictionnaire"""
    t = []
    for f in tf:
        for a in ta:
            if a['idArtiste'] == f['idArtiste']:
                t.append(fusion(f, a))
    return t
```

```python
t1 = jointure_1(t_film, t_artiste)
print(len(t1))
pprint(t1[0])
```

```
233
{'année': '1977',
 'codePays': 'US',
 'genre': 'Aventure',
 'idFilm': '11',
 'nom': 'Lucas',
 'prénom': 'George',
 'résumé': 'Il y a bien longtemps, dans une galaxie très lointaine... La '
           "guerre civile fait rage entre l'Empire galactique et l'Alliance "
           "rebelle. Capturée par les troupes de choc de l'Empereur menées par "
           'le sombre et impitoyable Dark Vador, la princesse Leia Organa '
           'dissimule les plans de l'Étoile Noire, une station spatiale '
           'invulnérable, à son droïde R2-D2 avec pour mission de les remettre '
           'au Jedi Obi-Wan Kenobi. Accompagné de son fidèle compagnon, le '
           "droïde de protocole C-3PO, R2-D2 s'échoue sur la planète Tatooine "
           'et termine sa quête chez le jeune Luke Skywalker. Rêvant de '
           'devenir pilote mais confiné aux travaux de la ferme, ce dernier se '
           'lance à la recherche de ce mystérieux Obi-Wan Kenobi, devenu '
           'ermite au cœur des montagnes désertiques de Tatooine...',
 'titre': 'La Guerre des étoiles'}
```

```python
# 2eme façon (plus élégante)

def jointure_2(tf, ta):
    """ Réalise la fusion entre tf et ta; renvoie un tableau de dictionnaire"""
    t = [fusion(f, a) for f in tf for a in ta if a['idArtiste'] == f['idArtiste']]
    return t
```

```python
t2 = jointure_1(t_film, t_artiste)
print(len(t2))
pprint(t2[0])
```

```
233
{'année': '1977',
 'codePays': 'US',
 'genre': 'Aventure',
 'idFilm': '11',
 'nom': 'Lucas',
 'prénom': 'George',
 'résumé': 'Il y a bien longtemps, dans une galaxie très lointaine... La '
           "guerre civile fait rage entre l'Empire galactique et l'Alliance "
           "rebelle. Capturée par les troupes de choc de l'Empereur menées par "
           'le sombre et impitoyable Dark Vador, la princesse Leia Organa '
           'dissimule les plans de l'Étoile Noire, une station spatiale '
           'invulnérable, à son droïde R2-D2 avec pour mission de les remettre '
           'au Jedi Obi-Wan Kenobi. Accompagné de son fidèle compagnon, le '
           "droïde de protocole C-3PO, R2-D2 s'échoue sur la planète Tatooine "
           'et termine sa quête chez le jeune Luke Skywalker. Rêvant de '
           'devenir pilote mais confiné aux travaux de la ferme, ce dernier se '
           'lance à la recherche de ce mystérieux Obi-Wan Kenobi, devenu '
           'ermite au cœur des montagnes désertiques de Tatooine...',
 'titre': 'La Guerre des étoiles'}
```

Résumé

Python possède une fonction de tri `sorted` qui retourne un tableau trié, par défaut par ordre croissant. Pour trier un tableau de dictionnaires, on doit utiliser le paramètre `key` qui indique à la fonction `sorted` le nom d'une fonction qui retournera un ou des critères de tri.

Pour fusionner deux tables ayant les mêmes attributs on utilise simplement la concaténation (+) de tableaux. Si on a un attribut en commun, on utilise un algorithme adapté pour réaliser la jointure